

Course Description — Jakarta EE and Microservices 2026

Libellé anglais

Jakarta EE and Microservices — Domain-Driven Design and Hexagonal Architecture

Compétences visées

Concevoir des systèmes, des applications ou des solutions d'ingénierie complexes

- Schématiser conceptuellement les solutions
- Évaluer la pertinence de différentes solutions

Réaliser des systèmes, applications ou solutions d'ingénierie complexes

- Créer, choisir ou adapter un composant d'un système, d'une application ou d'une solution d'ingénierie complexe
- Tester et valider des composants d'un système, d'une application ou d'une solution d'ingénierie complexe
- Intégrer des composants dans des systèmes, applications ou solutions d'ingénierie plus complexes
- Valider un système, une application ou une solution d'ingénierie complexe

Agir avec une démarche scientifique et éthique

- Apporter les arguments principaux pour justifier les choix
- Adopter des méthodes permettant de résoudre un problème

Communiquer avec pertinence et efficacité

- Argumenter le bien fondé des choix

AAV — Acquis d'apprentissage visés

Upon completion of this course, students will be able to:

1. **Develop** enterprise applications using Jakarta EE core technologies: Servlets, JSP, JPA, CDI, and JAX-RS.
 2. **Design and implement** a complete RESTful API with error handling, bean validation, and API versioning.
 3. **Model** a business domain by applying Domain-Driven Design patterns: Value Objects, Aggregates, Domain Services, Domain Events, and Repository Interfaces.
 4. **Restructure** an application following hexagonal architecture (Ports & Adapters), isolating the pure domain layer from infrastructure concerns.
 5. **Decompose** a monolithic application into independent microservices with inter-service communication (MicroProfile Rest Client), fault tolerance (circuit breaker, retry, timeout), and health checks.
 6. **Deploy** Jakarta EE applications in containers (Docker/Podman) on an OpenLiberty application server.
 7. **Apply** database migration strategies (Flyway) and API versioning in an iterative development context.
-

Contenu de l'enseignement

The course is organised into 8 sessions of 6 hours each (3h lecture + 3h lab), split into two parts:

Part 1: Jakarta EE Fundamentals (24h)

- **Session 1 — Introduction to Jakarta EE:** Jakarta EE and MicroProfile ecosystem, development environment setup (OpenLiberty, Maven, Podman/Docker), first servlet application.
- **Session 2 — Servlets, JSP & MVC:** HTTP lifecycle and servlet API, request/response handling, JSP views with JSTL, MVC pattern.
- **Session 3 — Java Persistence API (JPA):** ORM mapping, entity lifecycle, JPQL, Criteria API, manual transaction management, Flyway database migrations.

- **Session 4 — CDI and Dependency Injection:** DI principles, CDI scopes and qualifiers, interceptors, decorators, declarative transactions (@Transactional), bean-managed transactions (UserTransaction).

Part 2: Advanced Jakarta EE & Microservices (24h)

- **Session 5 — JAX-RS and RESTful Services:** REST principles, JAX-RS annotations, JSON processing with JSON-B, exception handling, Bean Validation, MicroProfile Rest Client.
- **Session 6 — Domain-Driven Design (DDD):** Strategic patterns (Bounded Context, Context Map), tactical patterns (Aggregate, Value Object, Entity, Domain Service, Domain Event, Repository), refactoring from an anemic to a rich domain model, API versioning.
- **Session 7 — Hexagonal Architecture:** Ports & Adapters pattern, pure domain layer with zero framework dependencies, primary adapters (REST v1/v2, Web) and secondary adapters (JPA, CDI), separation of domain and infrastructure entities.
- **Session 8 — Microservices Architecture:** Microservices patterns (Database per Service, API Gateway, Circuit Breaker), service decomposition, synchronous and asynchronous inter-service communication, MicroProfile Fault Tolerance, Health Checks, Metrics, Docker Compose orchestration.

All sessions are built around a **running project**: a banking application (BankingApp) developed progressively with client management, account management, operations, and money transfers.

Prérequis

- **Java SE:** mastery of Java fundamentals (classes, interfaces, inheritance, collections, generics, lambdas, streams)
- **Databases:** basic SQL and relational modelling concepts
- **HTTP protocol:** understanding of request/response lifecycle and HTTP methods (GET, POST, PUT, DELETE)
- **Object-oriented design:** SOLID principles, common design patterns (Factory, Observer, Strategy)
- **Tooling:** basic use of Maven and Git

Méthodes pédagogiques

- **Project-based learning:** all lab sessions are built around the same banking application, developed incrementally from Session 1 through Session 8.
 - **Progressive approach:** each concept builds upon the outcomes of the previous session; lab code from one session serves as the starting point for the next.
 - **Systematic practice:** every lecture concept is immediately put into practice in the associated lab, with a **starter/** codebase for students and a complete **solution/** for reference.
 - **Learning by example:** lectures use annotated code examples, architecture diagrams (Mermaid), and live demonstrations.
 - **Guided autonomy:** students work independently on the **starter/** code, using automated deployment scripts (Podman/Docker) to validate their progress at each step.
-

Modalités d'évaluation

Assessment	Format	Weight	Notes
Final exam	Written examination (closed book)	60 %	Covers all theoretical and architectural concepts from Sessions 1–8
Final project	Complete banking application (microservices architecture) with oral defence	40 %	Covers code quality, correct application of DDD and hexagonal patterns, and ability to argue technical decisions

The final project assessment criteria include:

- Code quality and adherence to best practices (Clean Code, SOLID)
 - Correct application of DDD and hexagonal architecture patterns
 - Functional deployment of the application as microservices
 - Ability to explain and justify architectural decisions during the oral defence
-

Ressources et bibliographie

Official Documentation

- Jakarta EE Specifications: <https://jakarta.ee/specifications/>
- Jakarta EE Tutorial (current): <https://jakarta.ee/learn/docs/jakartaee-tutorial/current>
- Open Liberty Guides: <https://openliberty.io/guides/>
- MicroProfile: <https://microprofile.io/>

Reference Books

- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.
- Newman, S. (2021). *Building Microservices* (2nd ed.). O'Reilly.
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Online Resources

- Jakarta EE GitHub: <https://github.com/eclipse-ee4j>
- Microservices Patterns: <https://microservices.io/patterns/>
- DDD Community: <https://www.domainlanguage.com/>